## Functional Dependency

if $X \to Y$, (Y is functionally dependent on X)

$t_1[X] = t_2[X]$

iff,

$t_1[Y] = t_2[Y]$

* $X \to Y$ need not imply $Y \to X$

(Instance of R) $r \to R$

| X | Y |
|---|---|
| $x_1$ | $y_1$ |
| $x_1$ | $y_1$ |
| $x_2$ | $y_2$ |
| $x_2$ | $y_2$ |
| $x_3$ | $y_2$ |
| $x_4$ | $y_3$ |

$t_1[X] = t_2[X]$ ($x_1$, $x_1$) Also $t_1[Y] = t_2[Y]$ ($y_1$, $y_1$)

$t_1[X] \neq t_2[X]$

Similarly we can check all tuples

so $X \to Y$ (X determines Y)

But Here $Y \to X$ is NOT true

## Armstrong Axioms

i) Reflexivity Rule — If $B \subseteq A$, then $A \to B$

e.g.- $\{S_{no}\} \subseteq \{S_{no}, S_{name}\}$

$\therefore \{S_{no}, S_{name}\} \to S_{no}$

Let, $Y \subseteq X$

and, $t_1[X] = t_2[X]$

so obviously, $t_1[Y] = t_2[Y]$

e.g.- $t_1[S_{no}, S_{name}] = t_2[S_{no}, S_{name}]$

So obviously, $t_1[S_{no}] = t_2[S_{no}]$

Hence, $\{S_{no}, S_{name}\} \to S_{no}$

ii) Augmentation Rule — If $A \to B$, then $A, C \to B, C$

e.g.- if $S_{no} \to city$

then $\{S_{no}, p_{no}\} \to \{city, p_{no}\}$

Let, $X \to Y$

i.e., $t_1[X] = t_2[X]$ iff $t_1[Y] = t_2[Y]$

$t_1[X, Z] = t_2[X, Z]$, then $t_1[Y, Z] = t_2[Y, Z]$

$\therefore \{X, Z\} \to \{Y, Z\}$

(iii) Transitivity Rule: If $X \to Y$ and $Y \to Z$, then $X \to Z$

e.g.- $S_{no} \to S_{name}$ and $S_{name} \to city$

then $S_{no} \to city$

Let, $A \to B$ and $B \to C$

$t_1[A] = t_2[A]$ Also, $t_1[B] = t_2[B]$

$\Rightarrow t_1[B] = t_2[B]$ $\Rightarrow t_1[C] = t_2[C]$

It is clear that,

$t_1[A] = t_2[B] \Rightarrow t_1[C] = t_2[C]$

(iv) Decomposition Rule: If $A \to B,C$, then $A \to B$ & $A \to C$

e.g.- if $S_{no} \to \{city, status\}$

then $S_{no} \to city$ & $S_{no} \to status$

Let, $A \to B, C$

$\because B \subseteq B,C$ and $C \subseteq B,C$

$\therefore B,C \to B$ and $B,C \to C$ (reflexivity rule)

Also, $A \to B, C$

$\therefore A \to B$ and $A \to C$ (transitivity rule)

(v) Union Rule: If $A \to B$ and $A \to C$, then $A \to B,C$

e.g.- if $S_{no} \to p_{no}$ and $S_{no} \to p_{name}$

then, $S_{no} \to \{p_{no}, p_{name}\}$

Let, $A \to B$ and, $A \to C$

$\Rightarrow AA \to AB$ $\Rightarrow AB \to BC$ (augmentation rule)

$\Rightarrow A \to AB$

$\therefore A \to BC$ (transitivity rule)

(vi) Pseudo-Transitivity Rule: If $A \to B$ holds and $BC \to D$ holds, then $AC \to D$

e.g.- if $S_{no} \to city$ and $\{city, status\} \to p_{no}$

then $\{S_{no}, status\} \to p_{no}$

Let, $A \to B$ and $BC \to D$

$\Rightarrow AC \to BC$ (augmentation rule)

$\therefore AC \to D$ (transitivity rule)

* Closure of an attribute is used to determine candidate keys.

Q) Find closure of the attributes

① R(A, B, C, D, E, F, G)

$A \rightarrow B$ $\quad \{A\}^+ = \{AB\}$ ; $\{B\}^+ = -$

$BC \rightarrow DE$ $\quad \{BC\}^+ = \{BCDE\}$

$AEF \rightarrow GC$ $\quad \{AEF\}^+ = \{AEFGCBD\}$ • Candidate key

② R(A, B, C, D, E)

$AB \rightarrow C$ $\quad \{AB\}^+ = \{ABC\}$

$CD \rightarrow E$ $\quad \{CD\}^+ = \{CDEB\}$

$DE \rightarrow B$ $\quad \{DE\}^+ = \{DEB\}$

$\{ADE\}^+ = \{A, D, E, B, C\}$

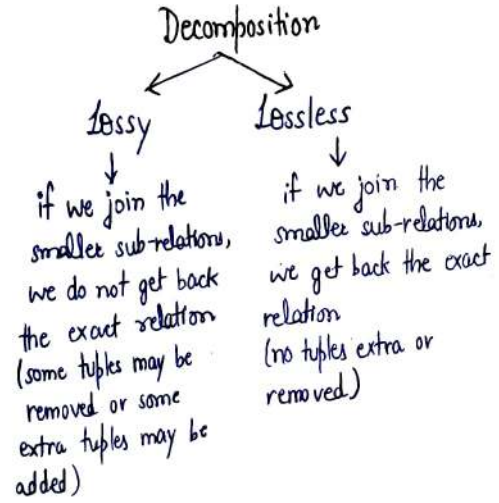
$\{ABD\}^+ = \{A, B, D, C, E\}$

$\{ACD\}^+ = \{A, C, D, E, B\}$

} Candidate Keys

## Decomposition

- dividing big relation into smaller sub-relations




e.g.-

R (eno, ename, pno, pname, plocation, hrs)

After decomposition:

$R_1 \rightarrow$ {eno, ename}

$R_2 \rightarrow$ {pno, pname, plocation}

$R_3 \rightarrow$ {eno, pno, hrs}

Functional Dependencies:

eno → ename

pno → {pname, ploc}

{eno, pno} → hrs

| | $a_1$ | $a_2$ | $a_3$ | $a_4$ | $a_5$ | $a_6$ |
|---|---|---|---|---|---|---|
| | eno | ename | pno | pname | plocation | hrs |
| $R_1$ | $a_1$ | $a_2$ | $b_{13}$ | $b_{14}$ | $b_{15}$ | $b_{16}$ |
| $R_2$ | $b_{21}$ | $b_{22}$ | $a_3$ | $a_4$ | $a_5$ | $b_{26}$ |
| $R_3$ | $a_1$ | $b_{32}$ $a_2$ | $a_3$ | $b_{34}$ $a_4$ | $b_{35}$ $a_5$ | $a_6$ |

- $a_i$ : in attribute present in relation
- $b_{ij}$ : $j^{th}$ attribute NOT present in $i^{th}$ relation

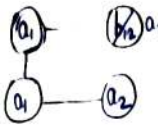
* If we get any row in the form of all a's, it is lossless (at any point)
* If we do not get any row in the form of all a's, it is lossless.

e.g.- eno → ename
pno → {pname, ploc}
{eno, pno} → hrs

$R_1$ (ename, plocation)
$R_2$ (eno, pno, hrs, pname, plocation)

| | $a_1$ | $a_2$ | $a_3$ | $a_4$ | $a_5$ | $a_6$ |
|---|---|---|---|---|---|---|
| | eno | ename | pno | pname | ploc | hrs |
| $R_1$ | $b_{11}$ | $a_2$ | $b_{13}$ | $b_{14}$ | $a_5$ | $b_{16}$ |
| $R_2$ | $a_1$ | $b_{22}$ | $a_3$ | $a_4$ | $a_5$ | $a_6$ |

Lossy decomposition




Why Normalization?
(i) remove redundancy
(ii) remove NULL constraint (attributes)
(iii) handle anomalies

Anomalies
- Updation anomalies
- Deletion anomalies
- Insertion anomalies

## Normal Forms

- 1NF → in a relation, all attributes must be single-valued and atomic in nature
- 2NF → all (1NF+) non-prime attributes is fully functionally dependent on primary key of that relation (supports (all candidate keys) xx trivial dependency)
- 3NF → if (2NF+) X → A, either X is primary key or A is prime attribute (trivial dependency NOT supported)
- BCNF (Boyce-Codd NF) → if (3NF+) X → A, X is super key and A is non-prime (non-trivial dependency)

e.g.- Dept (dno, dname, dmgrno, dlocation) (multi-valued) → 1NF form
Dept1 (dno, dname, dmgrno)
Dept2 (dno, dlocation)

dno → dname, dmgrno
dno → dlocation

{dno}+ = {dno, dname, dmgrno} (in Dept1)
∴ dno is Primary Key

e.g.- Emp-Project (eno, ename, {proj(pno, hr)}, city, status) → 1NF form + 2NF form
Emp-Project1 (eno, ename, city, status)
Emp-Project2 (eno, pno, hr)

eno → ename, city
city → status
{eno, pno} → hr

{eno}+ = {eno, ename, city, status}
∴ eno is Primary Key
{eno, pno}+ = {eno, pno, hr}
∴ {eno, pno} is Primary key

## Functional Dependency

- **Full**: if $X \to Y$, then if we remove any attribute from X and functional dependency doesn't exist
- **Partial**: if $X \to Y$, and removing any attribute from X doesn't affect functional dependency

## Functional Dependency

- **Trivial**
  - $X \to Y$ ← Non-trivial
  - $Y \to Z$ ← Non-trivial
  - $X \to Z$ ← Trivial (Transitive)
- **Non-trivial**

e.g. - SUP (sno, sname, pno, city, hrs)

$\{sno, sname\} \to city$ ← full (if we remove either sno or sname, dependency is no longer there)

$\{sno, pno\} \to hrs$ ← partial (if we remove sno, pno → hrs is still true)

$pno \to hrs$

## Attribute

- **Prime**: part of primary key
- **Non-prime**: not part of primary key

e.g. - Emp-Proj (eno, pno, ename, pname, plocation, hrs)

$\{eno, pno\} \to hrs$

$eno \to ename$

$pno \to \{pname, plocation\}$

$\{eno, pno\}^+ = \{eno, pno, hrs, ename, pname, plocation\}$

$\therefore \{eno, pno\}$ is Primary key

* NOT in 2NF form (as ename only depends on eno and, pname, ploc only depends on pno } Partial Dependencies

**2NF-form**

- Emp-Proj 1 (eno, pno, hrs)
- Emp-Proj 2 (eno, ename)
- Emp-Proj 3 (pno, pname, plocation)

e.g. - Emp-Proj (eno, ename, {Proj(pno, hr)}, city, status)

**3NF-form**

- Emp-Proj 1A (eno, ename, city)
- Emp-Proj 1B (city, status)
- Emp-Proj 2 (eno, pno, hr)

* All BCNF is 3NF but all 3NF is not BCNF

3NF: if $X \to A$, either X is primary key or A is prime attribute
BCNF: if $X \to A$, X is super key and A is non-prime attribute
} Non-trivial functional dependency (No transitivity)

e.g. – TEACH (student, course, instructor)

{student, course} → instructor

instructor → course

{student, course}$^+$ = {student, course, instructor}

- 3-NF but NOT BCNF

e.g. – LOTS (propertyId, cname, LOT#, area, price, taxrate)

propertyId → cname, LOT#, area, price, taxrate

{cname, LOT#} → {propertyId, area, price, taxrate}

cname → taxrate

area → price

- Decompose upto 3-NF.

{propertyId}$^+$ = {propertyId, cname, LOT#, area, price, taxrate}

∴ propertyId is a candidate key

{cname, LOT#}$^+$ = {propertyId, cname, LOT#, area, price, taxrate}

∴ {cname, LOT#} is also a candidate key

Let us assume propertyId to be our Primary key

- The relation is already in 1-NF as no multi-valued attributes are there
- The relation is also in 2-NF as all functional dependencies are fully dependent on primary key(all candidate keys)
- Also in 3-NF
- NOT in BCNF

e.g.- SUP(sno, sname, city, status, proj (pno, hrs))

sno → sname, city

city → status

sno, pno → hrs

. Decompose upto 3-NF

$\{sno\}^+ = \{sno, sname, city, status\}$

$\{sno, pno\}^+ = \{sno, sname, city, status, pno, hrs\}$

∴ {sno, pno} is a candidate key

Let us assume {sno, pno} be our primary key

1-NF

SUP1 (sno, sname, city, status)

SUP2 (sno, pno, hrs)

For SUP1, $\{sno\}^+ = \{sno, sname, city, status\}$

∴ sno. can be primary key for SUP1

For SUP2, $\{sno, pno\}^+ = \{sno, pno, hrs\}$

∴ {sno, pno} can be primary key for SUP2

2-NF

sno → sname, ~~city~~

sno → city

city → status

A/c transitivity rule,

sno → status

Hence, SUP1 is in 2-NF.

{sno, pno} → hrs

Hence, SUP2 is in 2-NF

3-NF

sno → sname

sno → city

city → status (NOT allowed)

SUP1A (sno, sname, city)

SUP1B (city, status)

Now, SUP1A and SUP1B are in 3-NF

{sno, pno} → hrs

∴ SUP2 is in 3-NF

e.g. – EMP (empid, branchno, branchaddress, name, position, hrs)

empid → {name, position}

branchno → {branchaddress}

{empid, branchno} → hrs

Decompose upto 3-NF

1-NF

- Since, there are no multi-valued attributes, EMP is already in 1-NF

{empid}$^+$ = {empid, name, position}

{empid, branchno} = {empid, branchno, name, position, branchaddress, hrs}

∴ {empid, branchno} is PK

2-NF

empid → {name, position}

∵ This is partial dependency

∴ EMP is not in 2-NF

EMP1 (empid, name, position)

EMP2 (empid, branchno, hrs)

EMP3 (branchno, branchaddress)

Now, these are in 2-NF

3-NF

EMP1, EMP2, EMP3 also in 3-NF (also in BCNF)

e.g. – LOTS (propertyId, cname, LOT#, area, price, taxrate)

propertyId → cname, LOT#, area, price, taxrate

{cname, LOT#} → propertyId, area, price, taxrate

cname → taxrate

area → price

Decompose upto 3-NF

- Already in 1-NF

propertyId, {cname, LOT#} are candidate keys

{propertyId}$^+$ = {propertyId, cname, LOT#, area, price, taxrate}

{cname, LOT#}$^+$ = {propertyId, cname, LOT#, area, price, taxrate}

2-NF

{cname, LOT#} → taxrate

but, cname → taxrate

Here, taxrate Not fully functionally dependent on {cname, LOT#}

LOT1 (propertyId, cname, LOT#, area, price)

LOT2 (cname, taxrate)

3-NF

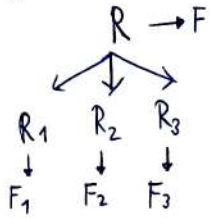
In LOT1, area → price, so NOT in 3-NF

LOT 1A (propertyId, cname, LOT#, area)

LOT 1B (area, peice)

· LOT2 already in 3-NF

* Dependency Preserving Decomposition




if $\pi F_1 \cup \pi F_2 \cup \pi F_3 = F$, then it is dependency preserving decomposition.

Q) A relational schema R(A, B, C, D, E, F) has the following functional dependencies:

$A \rightarrow CD$

$B \rightarrow C$

$F \rightarrow DE$

$F \rightarrow A$

It is decomposed into three relations:

$R_1(A, B, C)$, $R_2(A, F, D)$, $R_3(E, F)$

Find out whether these decomposition is dependency preserving or not and lossless or lossy.

| $R_1(A, B, C)$ | $R_2(A, F, D)$ | $R_3(E, F)$ |
|---|---|---|
| $A \rightarrow C$ | $F \rightarrow A$ | $F \rightarrow E$ |
| $B \rightarrow C$ | $F \rightarrow D$ | |
| | $A \rightarrow D$ | |

$\{A \rightarrow C \cup B \rightarrow C \cup F \rightarrow A \cup F \rightarrow D \cup A \rightarrow D \cup F \rightarrow E\}^+ = \{A \rightarrow CD, B \rightarrow C, F \rightarrow A, F \rightarrow DE\}$

∴ Closure of this union and closure of $\{F\}^+$ is same, it is dependency preserving.

| | $a_1$ | $a_2$ | $a_3$ | $a_4$ | $a_5$ | $a_6$ |
|---|---|---|---|---|---|---|
| | A | B | C | D | E | F |
| R1 | $a_1$ | $a_2$ | $a_3$ | $b_{14}$ $a_4$ | $b_{15}$ | $b_{16}$ |
| R2 | $a_1$ | $b_{21}$ | $b_{23}$ $a_3$ | $a_4$ | $b_{25}$ $a_5$ | $a_6$ |
| R3 | $b_{31}$ $a_1$ | $b_{32}$ | $b_{33}$ | $b_{34}$ $a_4$ | $a_5$ | $a_6$ |

$A \to CD$ $R_1(A,B,C)$
$B \to C$ $R_2(A,F,D)$
$F \to DE$ $R_3(E,F)$
$F \to A$

$\therefore$ It is lossy.

Q) $R(A,B,C,D,E,F,I,J)$

$AB \to C$
$A \to DE$
$B \to F$
$F \to GH$
$D \to IJ$

Find key of the relation and whether R in 3NF or not.

$\{AB\}^+ = \{A,B,C,D,E,F,G,H,I,J\}$

$\therefore \{AB\}$ is the Primary Key (a/c closure)

① R is already in 1-NF

② For 2-NF,

$AB \to DE$
$B \to F$
} Not fully functionally dependent on primary key, so NOT in 2-NF

$R_1(\underline{A,B}, C)$
$R_2(\underline{A}, D, E, I, J)$
$R_3(\underline{B}, F, G, H)$

$R_1$: $AB \to C$
~~So A~~
$R_2$: $A \to DE$
So, $A \to D$
$A \to E$
Also, $D \to IJ$
$D \to I$
$D \to J$
} decomposition rule
So, $A \to I$
$A \to J$
} transitive rule

③ For 3-NF,

$R_1(\underline{A,B}, C)$
$R_2a(\underline{A}, D, E)$
$R_2b(\underline{D}, I, J)$
$R_3a(\underline{B}, F)$
$R_3b(\underline{BF}, G, H)$

$R_3$: $B \to F$
$F \to GH$
So, $F \to G$
$F \to H$
} decomposition rule
and, $B \to G$
$B \to H$
} transitivity rule

## Multi-valued Dependency

$R(X, Y, Z)$

$X \twoheadrightarrow Y$ (X multi-valued determines Y)

$Y \twoheadrightarrow Z$

$t_1[X] = t_2[X] = t_3[X] = t_4[X]$

$t_1[Y] = t_3[Y]$ and, $t_4[Y] = t_2[Y]$

$t_3[Z] = t_2[Z]$ and, $t_1[Z] = t_4[Z]$

EMP(ename, pname, dname)

| ename | pname | dname |
|---|---|---|
| abc | X | aaa |
| abc | Y | bbb |
| abc | X | bbb |
| abc | Y | aaa |

* $X \twoheadrightarrow Y$ called "trivial multi-valued dependency" if Y is subset of X or $X \cup Y = R$, otherwise it is non-trivial multi-valued dependency.

## 4-NF

- A relational schema 'R' is in 4-NF w.r.t to set of dependencies for every non-trivial functional dependencies (multi-valued) $X \twoheadrightarrow Y$ in $\{F\}^+$, X is a super key

EMP (ename, pname, dname)

EMP1 (ename, pname)

| ename | pname |
|---|---|
| abc | X |
| abc | Y |

EMP2 (ename, dname)

| ename | dname |
|---|---|
| abc | bbb |
| abc | aaa |

Join Dependency: $\pi R_1 \bowtie \pi R_2 \bowtie \ldots \bowtie \pi R_n = R$

## 5-NF

- A relational schema 'R' is in the 5-NF w.r.t. functional dependencies set F which are multi-valued in nature and for every non-trivial join dependency $*(R_1, R_2, \ldots, R_n)$ in $\{F\}^+$, every $R_i$ is the super key of R

# Transaction

- Set of operations to perform a certain task.

## ACID Properties

- A → Atomicity
- C → Consistency
- I → Isolation
- D → Durability

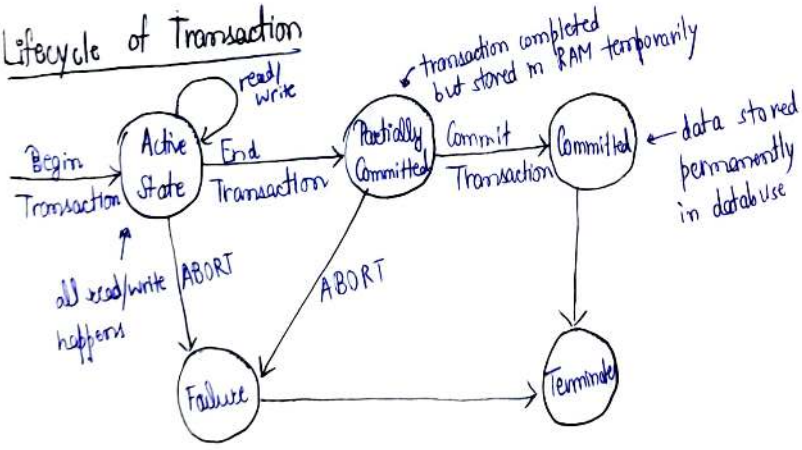
- Atomicity: either all steps of transaction is performed or none at all

(i) read(A)
(ii) A := A + 100
(iii) Write(A)
(iv) read(B)
(v) B := B + 100
(vi) write(B)

} either all 6 steps occur or none at all

- Consistency: each transaction takes the database from one consistent state to another consistent state
- Isolation: if multiple transactions are running simultaneously, all transactions should be isolated from all others
- Durability: after a transaction, the consistent state of the database must persist for some time

## Lifecycle of Transaction




Types of Transaction:

(i) Serial schedule — one transaction runs after another
(ii) Concurrent schedule — more than one transaction runs simultaneously.

- In serial schedule, if I/O operation comes, CPU goes idle

* Advantages of Concurrent Schedule:
  (i) increases system throughput
  (ii) reduces average response time

* Disadvantages of Concurrent Schedule:
  (i) Lost update

| $T_1$ | $T_2$ |
|---|---|
| read (A) | |
| A := A − 100 | |
| | read (A) |
| | R := A * 0.2 |
| | A := A + R |
| Write (A) | |
| read (B) | |
| | Write (A) |
| B := B + 100 | |
| Write (B) | |

here, this update is lost

(ii) Dirty read

| $T_1$ | $T_2$ |
|---|---|
| read (A) | |
| A := A − 100 | |
| write (A) | |
| | read (A) ← here, this is a dirty read |
| | A := A + 200 |
| | write (A) |
| read (B) | |
| $T_1$ fails | |

(iii) Unpredictable read

| $T_1$ | $T_2$ |
|---|---|
| read (A) | |
| | read (A) |
| | A := A + 100 |
| | write (A) |
| read (A) | |

same variable being read as different values after some time

# Scheduling

i) Serial Scheduling — after one transaction ends, another transaction will start

ii) Non-Serial Scheduling

- If we have $T_1, T_2, \ldots, T_n$ transactions running parallely and gives output like serial scheduling, they're called serializability.

Serializable
- Conflict Serializability
- View Serializability

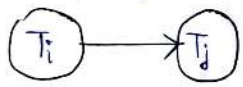
- We draw "precedence graph" to determine whether it is equivalent to serial schedule
- If graph doesn't gives a cycle, it is "conflict serializable".

## Conflict Serializability

① If $T_j$ → read operation (read(A))
after
$T_i$ → write operation (write(A))




② If $T_j$ → write operation (write(A))
after
$T_i$ → read operation (read(A))

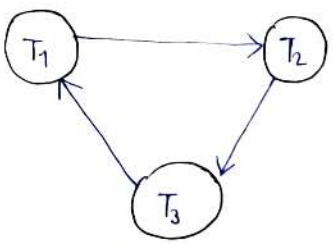



③ If $T_j$ → write operation (write(A))
after
$T_i$ → write operation (write(A))




e.g. -

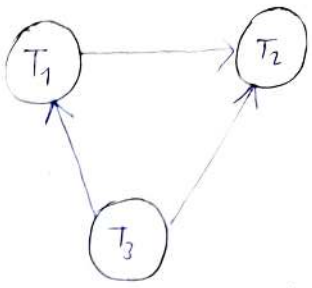
| $T_1$ | $T_2$ | $T_3$ |
|---|---|---|
| | read(Z) | |
| | read(Y) | |
| | write(Y) | |
| read(X) | | read(Y) |
| | | read(Z) |
| write(X) | | write(Y) |
| | read(X) | write(Z) |
| read(Y) | | |
| write(Y) | write(X) | |




∵ It forms a cycle, it is NOT equivalent to serial.

e.g.-

| $T_1$ | $T_2$ | $T_3$ |
|---|---|---|
| | | read(Y) |
| | | read(Z) |
| read(X) | | |
| write(X) | | |
| | | write(Y) |
| | | write(Z) |
| | read(Z) | |
| read(Y) | | |
| write(Y) | | |
| | read(X) | |
| | write(X) | |
| | read(Y) | |
| | write(Y) | |




∵ It doesn't form a cycle

∴ It is equivalent to serial schedule, hence it is "conflict serializable".

$T_1$ : $R_1(P)$ ; $R_1(R)$ ; $W_1(P)$

$T_2$ : $R_2(R)$ ; $R_2(Q)$ ; $W_2(Q)$ ; $W_2(P)$

$T_3$ : $R_3(P)$ ; $R_3(Q)$ ; $W_3(Q)$

$S_1$ : $R_1(P), R_2(R), R_1(R), R_1(P), R_3(Q), W_1(P), W_3(Q), R_2(Q), W_2(Q), W_2(P)$

| $T_1$ | $T_2$ | $T_3$ |
|---|---|---|
| read(P) | | |
| | read(R) | |
| read(R) | | |
| read(P) | | |
| | | read(Q) |
| write(P) | | |
| | | write(Q) |
| | read(Q) | |
| | write(Q) | |
| | write(P) | |

## View Serializable

* Two schedules S and S' are said to be view serializable if they are satisfying following conditions:

(i) same set of transactions participate in both S and S'

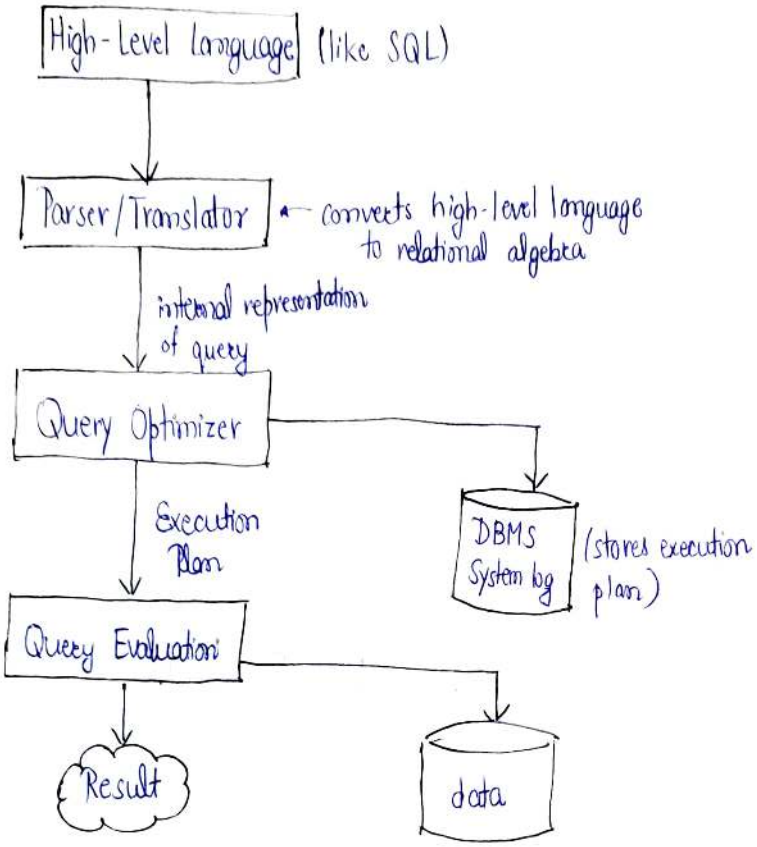
(ii) if the transaction $T_i$ reads initial value of a data item Q in schedule S, then $T_i$ must read the same value of data Q in schedule S'.

(iii) for each data item Q, if $T_i$ executes read(Q) operation after write(Q) operation of $T_j$ in schedule S, then $T_i$ must execute read(Q) operation after write(Q) operation of $T_j$ in schedule S'.

(iv) if $T_i$ performs final write operation in any data item Q in schedule S, then it must perform final write operation on same data item Q in schedule S'.

# Query Optimization

## Query Processing:




## Query Tree:

e.g. - $\Pi_{pname, bookTitle, aname}\left(\sigma_{price>10}(Book)\right)$

Tree: $\Pi_{pname, bookTitle, aname}$ (created before passing to optimizer)

↑

$\sigma_{price>10}$

↑

Book

- Query Optimization
  - Cost-based → we need to create query trees for each representation of query and then compute cost for each (large computational overhead)
  - Heuristic-based

# Equivalence Rules

Rule 1: $\sigma_{c_1 \wedge c_2}(R) = \sigma_{c_1}(\sigma_{c_2}(R))$

Rule 2: $\sigma_{c_1}(\sigma_{c_2}(R)) = \sigma_{c_2}(\sigma_{c_1}(R))$ (Commutative)

Rule 3: $\Pi_{a_1}(\Pi_{a_2}(\ldots(\Pi_{a_n}(R))\ldots)) = \Pi_{a_1}(R)$

Rule 4: $\Pi_{a_1, a_2, \ldots, a_n}(\sigma_c(R)) = \sigma_c(\Pi_{a_1, a_2, \ldots, a_n}(R))$

Rule 5: $\sigma_c(R_1 \times R_2) = R_1 \bowtie_c R_2$

$\sigma_{c_1}(R_1 \bowtie_{c_2} R_2) = R_1 \bowtie_{c_1 \wedge c_2} R_2$

Rule 6: $R_1 \times R_2 = R_2 \times R_1$ (Commutative)

$R_1 \bowtie_c R_2 = R_2 \bowtie_c R_1$

Rule 7: $(R_1 \times R_2) \times R_3 = R_1 \times (R_2 \times R_3)$ (Associative)

$(R_1 \bowtie R_2) \bowtie R_3 = R_1 \bowtie (R_2 \bowtie R_3)$

Rule 8: $\sigma_{c_1}(R_1 \bowtie_c R_2) = (\sigma_{c_1}(R_1)) \bowtie_c R_2$

$\sigma_{c_1 \wedge c_2}(R_1 \bowtie_c R_2) = (\sigma_{c_1}(R_1)) \bowtie_c (\sigma_{c_2}(R_2))$

Rule 9: $\Pi_{a_1 \cup a_2}(R_1 \bowtie_c R_2) = (\Pi_{a_1}(R_1)) \bowtie_c (\Pi_{a_2}(R_2))$

$\Pi_{a_1 \cup a_2}(R_1 \bowtie_c R_2) = (\Pi_{a_1 \cup a_2}(\Pi_{a_1 \cup a_3}(R_1)) \bowtie_c (\Pi_{a_2 \cup a_3}(R_2)))$

Rule 10: $R_1 \cup R_2 = R_2 \cup R_1$ (Commutative)

$R_1 \cap R_2 = R_2 \cap R_1$

$(R_1 \cup R_2) \cup R_3 = R_1 \cup (R_2 \cup R_3)$ (Associative)

$(R_1 \cap R_2) \cap R_3 = R_1 \cap (R_2 \cap R_3)$

Rule 11: $\sigma_c(R_1 \cup R_2) = \sigma_c(R_1) \cup \sigma_c(R_2)$

$\sigma_c(R_1 \cap R_2) = \sigma_c(R_1) \cap \sigma_c(R_2)$

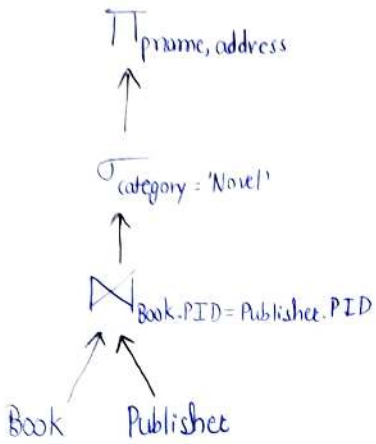
$\sigma_c(R_1 - R_2) = \sigma_c(R_1) - \sigma_c(R_2)$

Rule 12: $\Pi_a(R_1 \cup R_2) = \Pi_a(R_1) \cup \Pi_a(R_2)$

e.g.- $\Pi_{pname, address}\left(\sigma_{category = 'Novel'}(Book \bowtie_{Book.PID = Publisher.PID} Publisher)\right)$

Steps: SQL → Relational Algebra → Query Tree → Equivalent Relational Algebra → Optimized Query Tree

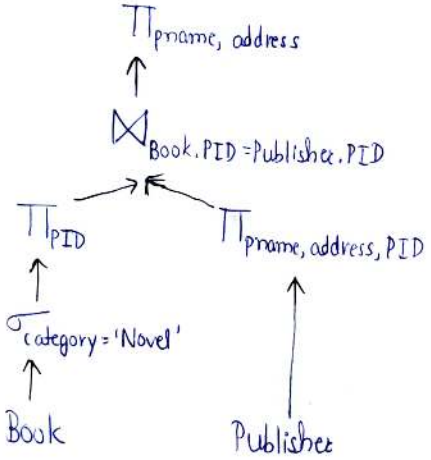
Query Tree:




* For optimization, we do: $\sigma \rightarrow \Pi \rightarrow \bowtie$

Equivalent RA: $\Pi_{pname, address}\left(\left(\Pi_{PID}\left(\sigma_{category = 'Novel'}(Book)\right)\right) \bowtie_{Book.PID = Publisher.PID} \left(\Pi_{pname, address, PID}(Publishers)\right)\right)$

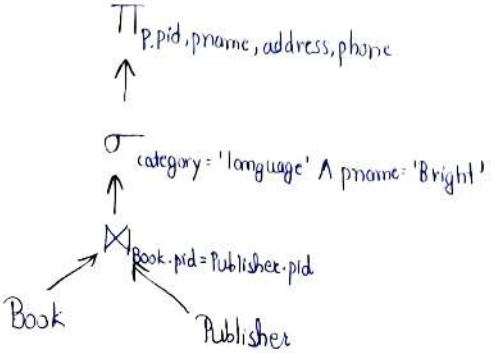
Optimized Query Tree:




e.g.- select p.pid, pname, address, phone from Book b, Publisher p where b.pid = p.pid and category = 'language' and pname = 'Bright';

RA: $\Pi_{P.pid, pname, address, phone}\left(\sigma_{category = 'language' \wedge pname = 'Bright'}\left(Book \bowtie_{Book.Pid = Publisher.Pid} Publisher\right)\right)$

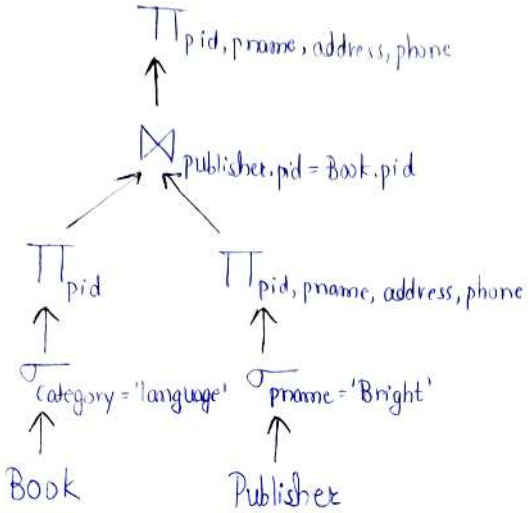
Query Tree:



Equivalent RA:

$$\Pi_{pid, pname, address, phone}\left(\left(\Pi_{pid}\left(\sigma_{category='language'}(Book)\right)\right) \bowtie_{Publisher.pid = Book.Pid} \left(\Pi_{pid, pname, address, phone}\left(\sigma_{pname='Bright'}(Publisher)\right)\right)\right)$$

Optimized Query Tree:




e.g. - select ISBN, booktitle, year, pagecount, price from Book b, Author a, Author-Book ab where b.ISBN = ab.ISBN and ab.aid = a.aid and aname = 'Ram';

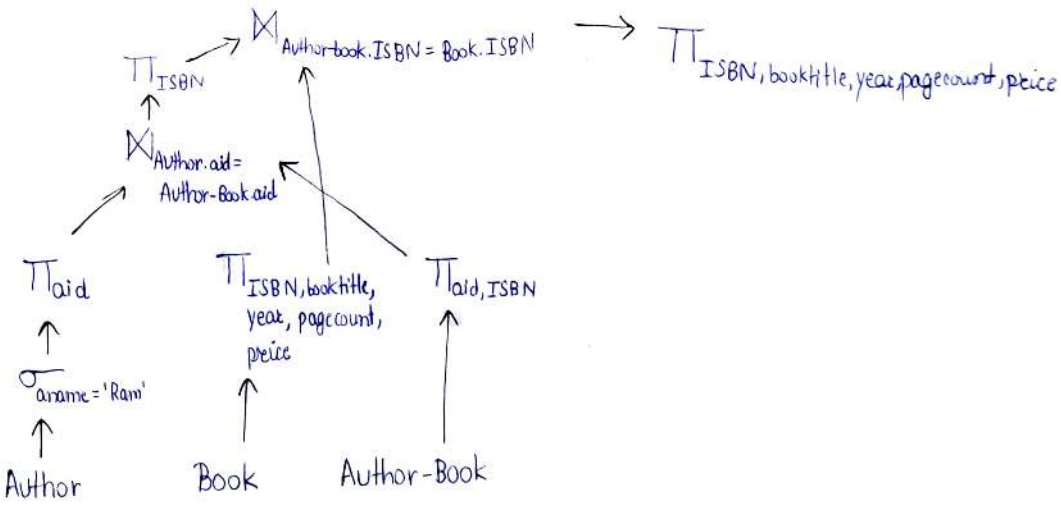
RA:

$$\Pi_{ISBN, booktitle, year, pagecount, price}\left(\sigma_{aname='Ram'}\left(\left(Book \underset{Book.ISBN = Author\text{-}Book.ISBN}{\bowtie} Author\text{-}book\right) \underset{Author.aid = Author\text{-}book.aid}{\bowtie} Author\right)\right)$$

Equivalent RA:

$$\Pi_{aid}\left(\sigma_{aname='Ram'}(Author)\right) \bowtie \Pi_{aid, ISBN}(Author\text{-}Book)$$

Optimized Query Tree:

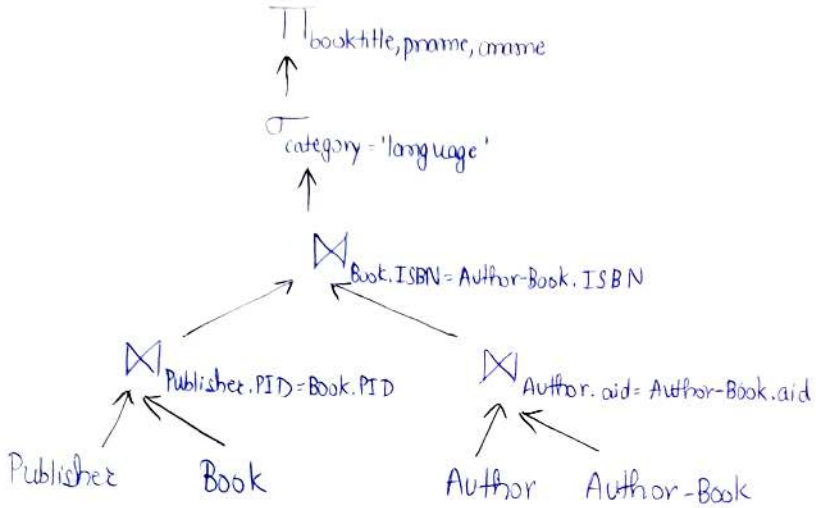



* We join those tables first in which we've 'σ' operation

e.g.— select booktitle, pname, aname from Book b, Publisher p, Author a, Author-Book ab where p.pid = b.pid and ab.aid = a.aid and b.ISBN = ab.ISBN and category = 'language';

RA: $\Pi_{booktitle, pname, aname}\left(\sigma_{category='language'}\left(\left(Publisher \bowtie_{Publisher.pid=Book.pid} Book\right) \bowtie_{Book.ISBN=Author\text{-}Book.ISBN} (Author\text{-}Book \bowtie_{Author.aid=Author\text{-}Book.aid} Author)\right)\right)$

Query Tree:




Equivalent RA:

Optimized Query Tree:

